



UNIVERSIDAD VERACRUZANA
FACULTAD DE NEGOCIOS Y TECNOLOGÍAS
LICENCIATURA EN INGENIERÍA DE SOFTWARE

REPORTE DE PROYECTO

Sistema de Gestión de Bandas (Backend REST)

Estudiante:

José Pablo Demeneghi
Victoria

Experiencia Educativa:

Principios de Diseño

Docente:

Adolfo Centeno Téllez

1. Introducción

El "Sistema de Gestión de Bandas" es una plataforma integral diseñada para la interacción entre entusiastas de la música, permitiendo la gestión de perfiles de bandas, sistemas de reacciones (likes/dislikes) y foros de comentarios en tiempo real.

Arquitectónicamente, el proyecto se rige bajo el principio de **Separación de Responsabilidades (SoC)**, dividiéndose en tres capas fundamentales:

1. **Capa de Persistencia:** Basada en PostgreSQL y desplegada en Aiven, garantizando la integridad de los datos.
2. **Capa de Lógica de Negocio (Backend):** Desarrollada con Spring Boot, encargada de centralizar las reglas de negocio y la seguridad mediante JWT.
3. **Capa de Presentación:** Una aplicación móvil y web construida en Flutter que consume los servicios REST del backend.

Para asegurar que el backend no sea simplemente un conjunto de funciones aisladas, sino un sistema mantenible y testeable, se han implementado patrones de diseño del *Gang of Four*. Estos patrones permiten resolver problemas recurrentes de creación, estructura y comunicación, elevando la calidad técnica del producto final.

2. Patrones Creacionales

Los patrones creacionales abstraen el proceso de instanciación, ocultando cómo los objetos se crean y se combinan.

2.1. Builder (Constructor)

Descripción: Este patrón permite construir objetos complejos paso a paso. Es ideal cuando un objeto tiene muchos atributos opcionales o requiere una configuración detallada antes de ser utilizado.

Aplicación: En el proyecto, se utiliza para generar los tokens JWT. Dado que un token requiere claims personalizados (como roles), fechas de expiración y algoritmos de firma, el *Builder* de la librería `io.jsonwebtoken` permite configurar el token de forma fluida y segura.

```
public String generateJwtToken(Authentication authentication) {
    UserDetailsImpl userPrincipal = (UserDetailsImpl) authentication.
        getPrincipal();

    // El Builder permite configurar el token de forma declarativa
    return Jwts.builder()
        .setSubject(userPrincipal.getUsername())
        .claim("roles", roles)
        .setIssuedAt(new Date())
        .setExpiration(new Date((new Date()).getTime() +
            jwtExpirationMs))
        .signWith(getSigningKey(), SignatureAlgorithm.HS256)
```

```
        .compact(); // Finaliza la construccion del String immutable
    }
```

Listing 1: Implementación de Builder para la generación de JWT

2.2. Singleton (Instancia Única)

Descripción: Garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.

Aplicación: Spring Boot utiliza este patrón por defecto para todos sus *Beans*. Componentes como `JwtUtils` o `SecurityUtils` se instancian una sola vez al arrancar el servidor. Esto es crítico para el rendimiento, ya que evita crear miles de objetos idénticos que saturen la memoria RAM durante las peticiones concurrentes.

```
@Component
public class JwtUtils {
    @Value("${pablo.app.jwtSecret}")
    private String jwtSecret;

    // Solo existe una instancia de esta clase para toda la aplicacion
    public boolean validateJwtToken(String authToken) { ... }
}
```

Listing 2: Singleton gestionado por el contenedor de Spring (@Component)

2.3. Factory Method (Método de Fábrica)

Descripción: Proporciona una interfaz para crear objetos en una superclase, permitiendo que las subclases o métodos estáticos decidan qué clase instanciar.

Aplicación: Se aplicó en la clase `UserDetailsImpl`. El sistema recibe una entidad `User` de la base de datos y necesita transformarla en un objeto compatible con Spring Security. El método `build()` actúa como una fábrica que encapsula la lógica de conversión de roles.

```
public static UserDetailsImpl build(User user) {
    List<GrantedAuthority> authorities = user.getRoles().stream()
        .map(role -> new SimpleGrantedAuthority(role.getName().name()))
        .collect(Collectors.toList());

    return new UserDetailsImpl(
        user.getId(), user.getUsername(),
        user.getEmail(), user.getPassword(),
        authorities);
}
```

Listing 3: Factory Method para transformar entidades de usuario

3. Patrones Estructurales

Estos patrones se ocupan de cómo las clases y objetos se componen para formar estructuras más grandes.

3.1. Facade (Fachada)

Descripción: Proporciona una interfaz simplificada a un subsistema complejo de clases, librerías o frameworks.

Aplicación: `AuthController` es la fachada del sistema de seguridad. Para que el usuario inicie sesión, el cliente solo llama a un método, pero internamente la fachada interactúa con el `AuthenticationManager`, el `SecurityContext` y el generador de tokens, ocultando esa complejidad al desarrollador del frontend.

```
@PostMapping("/signin")
public ResponseEntity<?> authenticateUser(@Valid @RequestBody LoginRequest
    loginRequest) {
    // Interaccion con subsistemas complejos de seguridad
    Authentication authentication = authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(loginRequest.
            getUsername(), loginRequest.getPassword()));

    SecurityContextHolder.getContext().setAuthentication(authentication);
    String jwt = jwtUtils.generateJwtToken(authentication);

    return ResponseEntity.ok(new JwtResponse(jwt, ...));
}
```

Listing 4: Patrón Facade simplificando el proceso de Login

3.2. Proxy (Representante)

Descripción: Proporciona un objeto sustituto para controlar el acceso al objeto original.

Aplicación: Se utiliza mediante la anotación `@PreAuthorize`. Spring crea un *Proxy* alrededor de los controladores. Cuando llega una petición, el Proxy intercepta la llamada, verifica si el usuario tiene el rol adecuado y, solo si es así, permite que la petición llegue al método real del controlador.

```
@DeleteMapping("/{id}")
@PreAuthorize("hasRole('USER') or hasRole('ADMIN')")
public ResponseEntity<?> eliminarBanda(@PathVariable Long id) {
    // El proxy valida el acceso antes de ejecutar este código
    bandaRepository.deleteById(id);
    return ResponseEntity.ok().build();
}
```

Listing 5: Proxy interceptor de seguridad para roles

4. Patrones de Comportamiento

Se centran en los algoritmos y la asignación de responsabilidades entre objetos.

4.1. Chain of Responsibility (Cadena de Responsabilidad)

Descripción: Permite pasar solicitudes a lo largo de una cadena de manejadores. Al recibir una solicitud, cada manejador decide si la procesa o si la pasa al siguiente.

Aplicación: Es el pilar del flujo HTTP en este proyecto. Las peticiones pasan por una cadena de filtros (`SimpleCorsFilter` → `AuthTokenFilter` → `DispatcherServlet`). Cada filtro valida una parte (CORS, Token, Rutas) y decide si el flujo continúa.

```
@Override
protected void doFilterInternal(HttpServletRequest request,
    HttpServletResponse response, FilterChain filterChain) {
    try {
        String jwt = parseJwt(request);
        if (jwt != null && jwtUtils.validateJwtToken(jwt)) {
            // Logica de validacion...
        }
    } catch (Exception e) {
        logger.error("Error de autenticacion: {}", e);
    }
    // Paso de la peticion al siguiente eslabon de la cadena
    filterChain.doFilter(request, response);
}
```

Listing 6: Eslabón de la cadena encargado de la validación del Token

5. Conclusión

El uso de patrones de diseño en el Sistema de Gestión de Bandas ha permitido pasar de un código funcional a un sistema de ingeniería profesional. La implementación de **Builder** y **Singleton** optimizó el manejo de recursos, mientras que **Facade**, **Proxy** y **Chain of Responsibility** blindaron la seguridad del sistema sin comprometer la limpieza del código. Esta estructura no solo facilita el mantenimiento actual, sino que garantiza que el sistema pueda escalar y adaptarse a futuros requerimientos con un impacto mínimo en el código existente.